CS2323 - Computer Architecture

Lab-4: Taylor Series Implementation using RISC-V F-extension

MA24BTECH11022

1 Design Choice

The design of the assembly code closely follows the following C++ snippet:

```
#include <stdio.h>

double exp(double x, int terms){
    double res = 1, t = 1;
    for(int i=2; i<=terms; i++){
        t = x/i;
        res = res+t;
    }
    return res;
}

double sin(double x, int terms){
    double res = x, t = x;
    for(int i=2; i<=terms; i++){
        t *= -(x*x)/((2*i-1)*(2*i-2));
        res = res+t;
    }
    return res;
}

double cos(double x, int terms){
    double res = 1, t = 1;
    for(int i=2; i<=terms; i++){
        t *= -(x*x)/((2*i-2)*(2*i-3));
        res = res+t;
    }
    return res;
}

double ln(double x, int terms){
    double res = x-1, t = 1;
    for(int i=2; i<=terms; i++){
        t *= -(x-1);
        res = res+t/i;
    }
    return res;
}

double fracx(double x, double terms){
    double res = 1, t = 1;
    for(int i=2; i<=terms; i++){
        t *= -(x-1);
        res = res+t;
    }
    return res;
}

int main(){
    int N;
    for(int i=0; i<N; i++){
        int code, terms;
        double x;
        switch(code){
            case 1:
                printf("%ld ", exp(x, terms));
                break;
            case 2:
                printf("%ld ", sin(x, terms));
                break;
            case 3:
                printf("%ld ", cos(x, terms));
                break;
            case 4:
                printf("%ld ", ln(x, terms));
                break;
            case 5:
                printf("%ld ", fracx(x, terms));
                break;
        }
    }
}
```

Each of the 5 taylor series are implemented in their own function blocks in a modular fashion. In `main`, the control is passed to a taylor series according to the code in the input. Since the Taylor Series of `ln(x)` about `x=0` does not exist, I have expanded `ln(x)` about `x=1`, and similarly so for `1/x`. The terms in the series are being computed in a running fashion, so there is no need for making helper functions `factorial()` and `pow()`.

2 Verification Process

I verified my logic and implementation using the C code as well as directly running `main.s`. I generated a bulk of test cases, that would check edge cases such as number of terms being negative, division by zero, etc.

3 Issues Faced

I faced a huge conundrum at the very beginning of this assignment. We had to use F extension in this assignment, but my choice of RISC-V simulator, RIPES did not have one. So I had to switch to KITES. After typing my code, I ran it and KITES showed the infamous `stoi` error. I swicthed from KITES to another simulator named RARS. Finally my code assembled without any error, and I proceeded with debugging my program.
Other than this the only issues faced were silly mistypings here and there. I chose my design for the control flow fairly quickly, so I did not spend much time there and proceeded with typing the assembly code.